

Análise Estática e Linter - Linguagem Imperativa 2

Paradigmas de Linguagens de Programação
Profº Augusto Sampaio
Nickolas Vitor,
Romulo Vital



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Visão Geral

- Partimos de uma base que já interpretava a imperativa 2
- Parser só reconhece a árvore. Semântica diz se é válido e o Linter monta a qualidade
- Isso evita misturar diagnóstico de erro com diagnóstico de estilo, que tem naturezas diferentes

```
Código-fonte
  ↓ JavaCC (Imperative2.jj)
Análise léxica + sintática
  ↓
                        AST
  ↓
Análise semântica
  ↓
Análise de linter
  ↓
Relatório de erros e avisos
```

Levantar e Entender a Base

- Antes de escrever o código, fomos entender o que já existia
- Quatro perguntas surgiram dentro dessa fase:
 1. Onde a AST é construída ?
 2. Como a execução atual consome essa AST ?
 3. Onde encaixamos a análise semântica ?
 4. Onde encaixamos o Linter ?

Mapa da Base do Código

- O parser constrói tudo a partir da classe raiz Programa
- A AST já tinha nó para praticamente toda a linguagem (If, While, Atribuição, Declarações, Expressões).
- O controle de escopo já existia em Contexto-<T> uma pilha de HashMap.

```
src/li2/plp/imperative2/
├── parser/
│   └── Imperative2.jj      ← parser JavaCC
├── Programa.java          ← raiz da AST
├── command/               ← IfThenElse, While,
Atribuicao...
│   ├── declaration/       ← DeclaracaoVariavel,
DeclaracaoProcedimento...
│   └── expressions2/      ← ExpBinaria, ExpUnaria,
Id, ValorInteiro...

expressions2/memory/Contexto.java ← escopo
legado (pilha de HashMap)
```

Pontos de extensão e Decisão de Arquitetura

- A base estava sólida para executar programas - Para análise estática identificamos dois pontos onde iria precisar estender:
 1. Expor a AST aos Visitors - Acrescentar nós para que um percorrer consiga descer pela árvore.
 2. Criar a infraestrutura de análise - Visitor, tabela de símbolos própria e classe de erro semântico.
 3. Getters Públicos a classes da AST
- O Visitor foi implementado com `instanceof` e não pelo padrão Visitor clássico com `accept()`

Estrutura da Análise Semântica

- Visitor Semântico
 - percorre a AST
- Se aparece erro, o programa não executa.

Parser



Visitor Semântico



└ se OK → `checaTipo` → execução

└ se erro → relatório, aborta

Tabela de Símbolos

- Tabela de símbolos própria
- Complementa o Contexto<Tipo>
- Classes que formam a tabela
 1. Simbolo - Entrada declarada (Nome, Tipo, Kind, Contadores)
 2. Escopo - LinkedHashMap e Ponteiro para o pai
 3. TabelaSimbolos - Orquestra os escopos (abrir, fechar, buscar)

Código Tabela de Símbolos

```
public class TabelaSimbolos {
    private Escopo escopoAtual;
    private final List<Escopo> escoposFechados = new ArrayList<>();
    private int proximoNivel = 0;

    public void abrirEscopo() {
        this.escopoAtual = new Escopo(proximoNivel++, escopoAtual);
    }

    public void fecharEscopo() {
        escoposFechados.add(escopoAtual); // ← não destrói: arquiva
        escopoAtual = escopoAtual.getPai();
    }

    public Simbolo buscar(String nome) {
        Escopo e = escopoAtual;
        while (e != null) {
            Simbolo s = e.buscarLocal(nome);
            if (s != null) return s;
            e = e.getPai(); // ← sobe a cadeia
        }
        return null;
    }
}
```


Visitor Semântico

- Despacho por InstanceOf
- Visitor trabalha com três frentes
 - Comandos
 - Declarações
 - Expressões
- Não para no primeiro erro
- Tipo desconhecido (null) suprime cascata

Código Visitor

```
public class VisitorSemantico {

    private final TabelaSimbolos tabela = new TabelaSimbolos();
    private final List<ErroSemantico> erros = new ArrayList<>();

    public List<ErroSemantico> analisar(Programa programa) {
        if (programa != null && programa.getComando() != null) {
            visitarComando(programa.getComando());
        }
        return Collections.unmodifiableList(erros);
    }

    private void visitarComando(Comando c) {
        if (c instanceof IfThenElse) { visitarIfThenElse((IfThenElse) c); return; }
        if (c instanceof While) { visitarWhile((While) c); return; }
        if (c instanceof Atribuicao) { visitarAtribuicao((Atribuicao) c); return; }
        if (c instanceof ChamadaProcedimento) { visitarChamadaProcedimento(...); return; }
        if (c instanceof ComandoDeclaracao) { visitarComandoDeclaracao(...); return; }
        if (c instanceof SequenciaComando) { /* visita os dois lados */ return; }
        if (c instanceof Read) { visitarRead((Read) c); return; }
        if (c instanceof Write) { tipoDe((Write) c).getExpressao(); return; }
    }
}
```

Linters

O que são linters?

Um linter é uma ferramenta de análise estática que examina o código-fonte sem executá-lo, identificando erros, inconsistências de estilo e padrões problemáticos automaticamente.



Analisa

Examina o código sem precisar executá-lo (análise estática)



Detecta

Erros de sintaxe, bugs potenciais e violações de estilo



Sugere

Correções e boas práticas antes do código ir para produção

Linters

Eslint

```
// .eslintrc.json
{ "rules": {
    "no-unused-vars": "warn",
    "eqeqeq": "error",
    "no-console": "warn",
  }
}
```

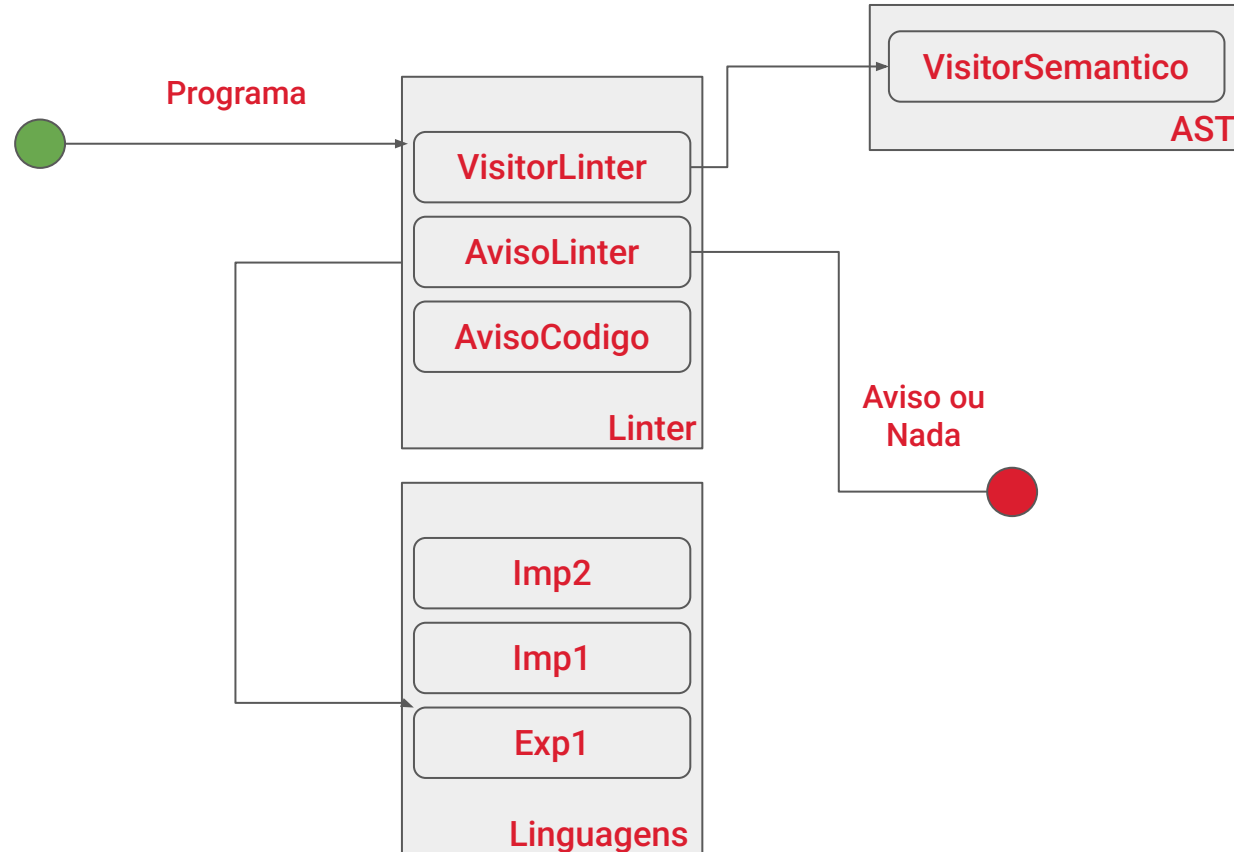
Pylint

```
# saída do Pylint

C0114: Missing module docstring
C0116: Missing function docstring
W0611: Unused import os
E1101: Module has no member 'X'
R0913: Too many arguments (7/5)

Your code has been rated at 6.5/10
```

Entendendo a estrutura montada



Avisos

```
1 package li2.plp.imperative2.linter;
2
3 public enum AvisoCodigo {
4     VAR_NAO_UTILIZADA,
5     PARAM_NAO_UTILIZADO,
6     CODIGO_MORTO_RAMO_THEN,
7     CODIGO_MORTO_RAMO_ELSE,
8     CODIGO_MORTO_WHILE,
9     COMPLEXIDADE_PROCEDIMENTO
10 }
11
```

```
public enum AvisoCodigo {
    VAR_NAO_UTILIZADA("no-unused-vars"),
    PARAM_NAO_UTILIZADO("no-unused-params"),
    CODIGO_MORTO_RAMO_THEN("no-dead-code-then"),
    CODIGO_MORTO_RAMO_ELSE("no-dead-code-else"),
    CODIGO_MORTO_WHILE("no-dead-code-while"),
    COMPLEXIDADE_PROCEDIMENTO("max-complexity");

    private final String codigo;
```

Variáveis Não Utilizadas

Ideia

Lê diretamente a TabelaSimbolos já preenchida pelo VisitorSemantico — sem percurso adicional na AST

O VisitorSemantico já incrementa contadores lido e escrito em cada Símbolo durante a análise.

Variáveis write-only
(escrito > 0 mas lido == 0)
também são capturadas.

```
private void verificarVariaveisNaoUtilizadas() {  
    for (Escopo escopo : tabela.getTodosEscopos()) {  
        for (Simbolo s : escopo.getSimbolos().values()) {  
            if (s.getKind() == SimboloKind.VARIAVEL && s.getLido() == 0) {  
                avisos.add(new AvisoLintar(  
                    AvisoCodigo.VAR_NAO_UTILIZADA,  
                    "variavel '" + s.getNome() + "' declarada mas nunca lida"));  
            } else if (s.getKind() == SimboloKind.PARAMETRO && s.getLido() == 0) {  
                avisos.add(new AvisoLintar(  
                    AvisoCodigo.PARAM_NAO_UTILIZADO,  
                    "parametro '" + s.getNome() + "' declarado mas nunca lido"));  
            }  
        }  
    }  
}
```

Código Morto

Ideia

Avalia expressões booleanas em tempo de compilação sem executar o programa

O linter continua percorrendo código morto para contar decisões, garantindo aviso de complexidade em código inalcançável

```
private void visitarIfThenElse(IfThenElse c) {  
    if (procedimentoAtual != null) {  
        complexidade++;  
    }  
    Expressao cond = c.getExpressao();  
    Boolean valor = avaliador.avaliar(cond);  
    if (Boolean.TRUE.equals(valor)) {  
        avisos.add(new AvisoLinter(AvisoCodigo.CODIGO_MORTO_RAMO_ELSE,  
            "ramo 'else' inalcançável porque a condicao e sempre 'true'"));  
    } else if (Boolean.FALSE.equals(valor)) {  
        avisos.add(new AvisoLinter(AvisoCodigo.CODIGO_MORTO_RAMO_THEN,  
            "ramo 'then' inalcançável porque a condicao e sempre 'false'"));  
    }  
    visitarComando(c.getComandoThen());  
    visitarComando(c.getComandoElse());  
}
```


Complexidade de procedimento

Ideia

Conta pontos de decisão (if,while) dentro do corpo de cada procedimento.

Somente o corpo do procedimento é contado — o programa principal não é avaliado

Limiar é 4, acima desse valor emite aviso

```
private void visitarDeclaracaoProcedimento(DeclaracaoProcedimento dp) {  
    String nomeProcAnterior = procedimentoAtual;  
    int complexidadeAnterior = complexidade;  
  
    procedimentoAtual = dp.getId().getIdName();  
    complexidade = 0;  
  
    visitarComando(dp.getDefProcedimento().getComando());  
  
    if (complexidade > LIMIAR_COMPLEXIDADE) {  
        avisos.add(new AvisoLintar(  
            AvisoCodigo.COMPLEXIDADE_PROCEDIMENTO,  
            "complexidade " + complexidade + " ultrapassa o limite de " + LIMIAR_COMPLEXIDADE,  
            procedimentoAtual));  
    }  
  
    procedimentoAtual = nomeProcAnterior;  
    complexidade = complexidadeAnterior;  
}
```

Pontos de melhoria

- ~~Mostrar linha na erro~~ (Filipe)
- ~~Código de erro padronizado com algum linter do mercado~~ (Filipe)
- Dar aviso do linter sempre que mudar o texto (Filipe)
- Arquivo de configuração para personalizar o linter por parâmetros
- Criar mais indicadores e para mais linguagens
- Adicionar erro de nó de comando não tratado

Exemplos

Imperativa 2

```
{  
  var naoUsada = 0,  
  proc muitoComplexo() {  
    if true then skip else skip;  
    if true then skip else skip;  
    if true then skip else skip;  
    if true then skip else skip;  
    if true then skip else skip  
  } ;  
  if true then write(1) else write(2);  
  call muitoComplexo()  
}
```

==== AVISOS DO LINTER ====

```
LINT [VAR_NAO_UTILIZADA]: variavel 'naoUsada' declarada mas nunca lida  
LINT [CODIGO_MORTO_RAMO_ELSE]: ramo 'else' inalcançavel porque a condicao e sempre 'true'  
LINT [CODIGO_MORTO_RAMO_ELSE]: ramo 'else' inalcançavel porque a condicao e sempre 'true'  
LINT [CODIGO_MORTO_RAMO_ELSE]: ramo 'else' inalcançavel porque a condicao e sempre 'true'  
LINT [CODIGO_MORTO_RAMO_ELSE]: ramo 'else' inalcançavel porque a condicao e sempre 'true'  
LINT [CODIGO_MORTO_RAMO_ELSE]: ramo 'else' inalcançavel porque a condicao e sempre 'true'  
LINT [COMPLEXIDADE_PROCEDIMENTO]: (em 'muitoComplexo') complexidade 5 ultrapassa o limite de 4  
LINT [CODIGO_MORTO_RAMO_ELSE]: ramo 'else' inalcançavel porque a condicao e sempre 'true'
```

Exemplos

Imperativa 2

```
{
  var x = 10,
  var w = 1,
  proc meuProcedimentoComplexo(int k) {
    if x == 10 then (
      skip
    ) else (
      skip
    );

    if x == 5 then (
      skip
    ) else (
      skip
    );

    while(x == 1) do (
      skip
    )
  };

  while(false) do (
    x := x + 1
  );

  call meuProcedimentoComplexo(1)
}; write(1+1)
```

==== AVISOS DO LINTER ====

LINT [PARAM_NAO_UTILIZADO]: parametro 'k' declarado mas nunca lido

LINT [VAR_NAO_UTILIZADA]: variavel 'w' declarada mas nunca lida

LINT [CODIGO_MORTO_WHILE]: corpo do 'while' inalcançavel porque a condicao e sempre 'false'